CT-159 · DATA STRUCTURES

Stacks & Recursion

Lecture 4 — LIFO structures, expression evaluation & recursive thinking

Presenter: Dr. Muhammad Kamran

SECTION 01

The Stack

A Last-In-First-Out structure — all the action happens at one end.

SECTION 01 · STACK

01

Stacks in the Real World

- Think of a stack of plates or a pile of books: you add and remove only from the top.
- You can't pull a plate from the middle without lifting the ones above it.
- A stack data structure works exactly the same way.

The last item you place on top is the first one you take off.





SECTION 01 · STACK

02

What is a Stack?

A stack allows data operations at one end only — the top.

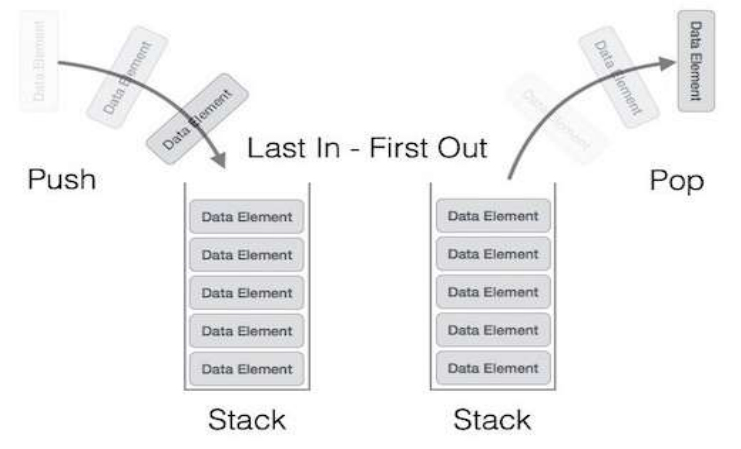
LIFO	Last-In, First-Out: the newest element is removed first.
Top only	At any time you can only access the top element.
Push	The operation that adds an element to the top.
Pop	The operation that removes the top element.

SECTION 01 · STACK

03

Stack Representation

Push adds to the top; pop removes from the top; everything else stays put.



SECTION 01 · STACK

04

Basic Operations

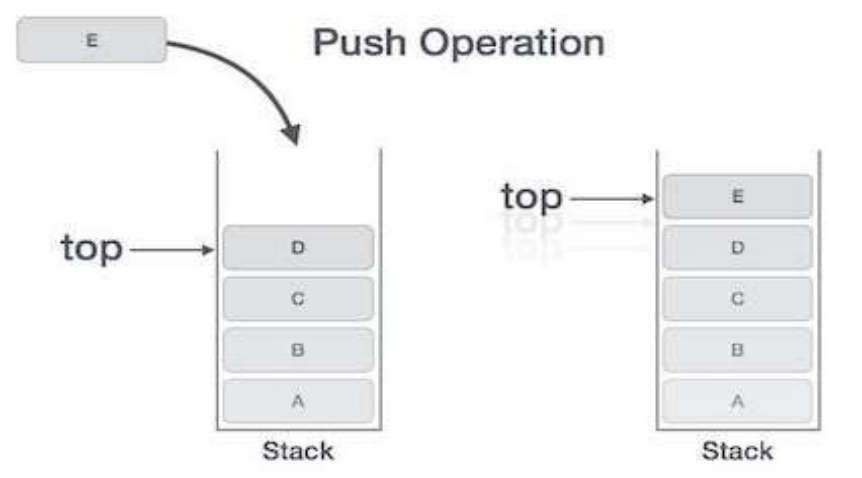
push(x)	Store an element x on top of the stack.
pop()	Remove and return the top element.
peek() / top()	Look at the top element without removing it.
isEmpty()	Check whether the stack has no elements.
isFull()	Check whether the stack has reached its capacity.

SECTION 01 · STACK

05

Push Operation

Add a new element on top — after checking the stack isn't full.



SECTION 01 · STACK

06

Push — Algorithm

Steps

1. Check if the stack is full
2. If full → overflow error, exit
3. Increment top by one
4. Place the new data at top

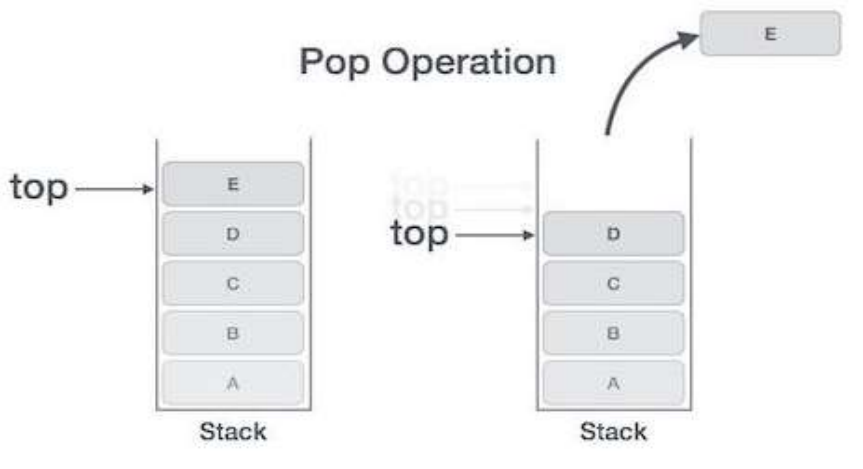
```
// pseudocode
procedure push(stack, data):
    if isFull(stack):
        return OVERFLOW
    top = top + 1
    stack[top] = data
end procedure
```

SECTION 01 · STACK

07

Pop Operation

Remove the top element — after checking the stack isn't empty.



SECTION 01 · STACK

08

Pop — Algorithm

Steps

1. Check if the stack is empty
2. If empty → underflow error, exit
3. Read the element at top
4. Decrement top by one
5. Return the read element

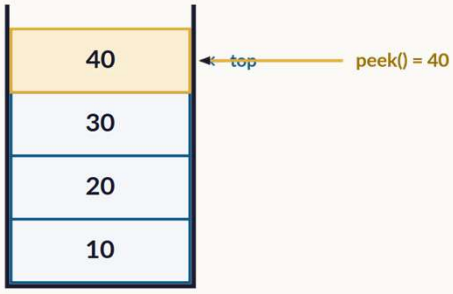
```
// pseudocode
procedure pop(stack):
    if isEmpty(stack):
        return UNDERFLOW
    data = stack[top]
    top = top - 1
    return data
end procedure
```

SECTION 01 · STACK

09

Peek Operation

Read the top value without changing the stack.



peek reads the top value without removing it — the stack is unchanged.

SECTION 01 · STACK

10

Implementing a Stack (Linked List)

push	Insert a new node at the beginning of the list — that node becomes the new top.
pop	Move top to top.next and free the old top node (if the list isn't empty).
peek	Return the data of the top node without removing it.
display	Start at top and traverse the list, printing each node.

SECTION 01 · STACK

11

Stack via Linked List (C++ sketch)

```
struct Node { int data; Node* next; };
Node* top = nullptr;

void push(int x) {                  // O(1)
    Node* n = new Node{ x, top };
    top = n;
}
int pop() {                         // O(1)
    if (top == nullptr) return -1; // underflow
    int x = top->data;
    Node* t = top; top = top->next;
    delete t;
    return x;
}
```

SECTION 01 · STACK

12

Applications of Stacks

- Function calls and recursion (the call stack).
- Undo / redo in editors.
- Expression evaluation and conversion (infix ↔ postfix).
- Browser history (back button).
- Checking balanced parentheses and syntax.

SECTION 02

Expression Evaluation

How stacks turn human-readable maths into something a computer can crunch.

Three Notations

Infix	Operator between operands — human-readable. e.g. (a + b) * c
Prefix (Polish)	Operator before operands. e.g. * + a b c
Postfix (Reverse Polish)	Operator after operands. e.g. a b + c *

SECTION 02 · EXPRESSIONS

14

Infix → Postfix Conversion

- Computers convert human infix into postfix first, then evaluate the postfix.
- Scan the infix left to right, using a stack to hold operators.
- Operands (a, b, c…) go straight to the output.
- An operator waits on the stack until a lower- or equal-precedence operator arrives.

SECTION 02 · EXPRESSIONS

15

Infix → Postfix — Algorithm

Steps

1. Operand → append to output
2. '(' → push onto the stack
3. ')' → pop to output until '('
4. Operator → pop while top has ≥ precedence, then push
5. End → pop all remaining operators

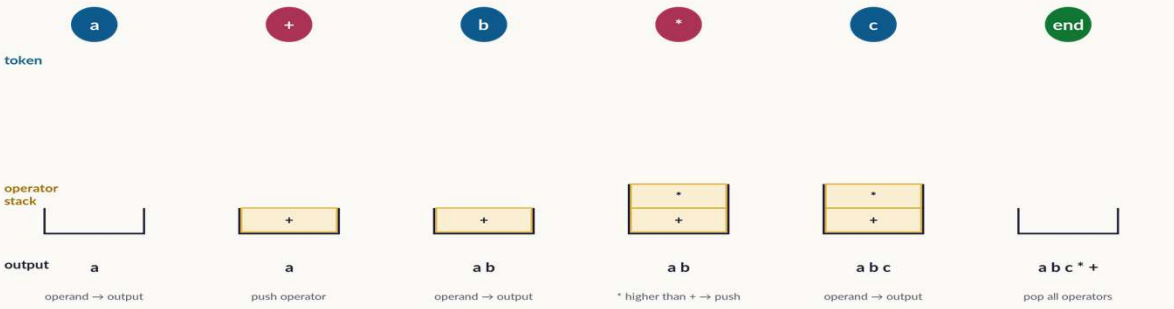
```
// pseudocode
for each token t in infix:
    if t is operand:
        output += t
    else if t == '(':
        push(t)
    else if t == ')':
        while top != '(':
            output += pop()
        pop()               // discard '('
    else:                   // operator
        while stack not empty and
              prec(top) >= prec(t):
            output += pop()
        push(t)
while stack not empty:
    output += pop()
```

SECTION 02 · EXPRESSIONS

16

Infix → Postfix — Walkthrough

*Converting a + b * c → a b c * + , one token at a time.*



Infix: a + b * c → Postfix: a b c * +

Operands go straight to output. Operators wait on a stack until a lower/equal-precedence operator arrives. On "(" push it; on ")" pop until the matching "(". At the end, pop every remaining operator.

SECTION 02 · EXPRESSIONS

17

Postfix Evaluation — Algorithm

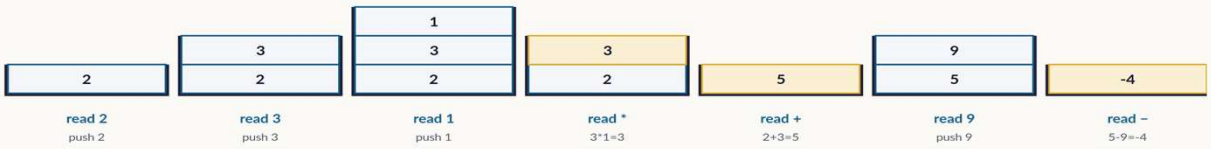
- Scan the expression from left to right.
- If the token is an operand (number) → push it onto the stack.
- If the token is an operator → pop the top two values, apply it, push the result.
- When scanning ends, the single value left on the stack is the answer.

SECTION 02 · EXPRESSIONS

18

Postfix Evaluation — Walkthrough

*The stack after each token of 2 3 1 * + 9 - → answer −4.*



Rule: number → push. Operator → pop two, apply, push the result.
Final answer left on the stack: −4

SECTION 02 · EXPRESSIONS

19

Next Greater Element (LeetCode 496) — Homework

Problem (in short): for each element, find the first element to its right that is greater than it (or −1 if none).

- Example: for [4, 1, 2] the next greater elements are [-1, 2, -1] within their own array.
- Naive way: for each element, scan everything to its right — that's $O(n^2)$.
- Stack way: keep a decreasing stack; when a bigger number appears, it's the answer for everything smaller waiting on the stack.
- This finds every answer in a single pass — O(n).

SECTION 02 · EXPRESSIONS

20

Next Greater Element — The Code

```
// nums2 holds the full array; answer[x] = next greater
unordered_map<int,int> nextGreater;
stack<int> st;

for (int x : nums2) {
    while (!st.empty() && st.top() < x) {
        nextGreater[st.top()] = x;   // x resolves it
        st.pop();
    }
    st.push(x);
}
while (!st.empty()) {                // nothing bigger
    nextGreater[st.top()] = -1;
    st.pop();
}
```

SECTION 02 · EXPRESSIONS

21

Next Greater Element — Why It Works

Monotonic stack	The stack always holds elements still waiting for a greater one, in decreasing order.
The trigger	A new bigger value pops and answers every smaller element waiting below it.
Leftovers	Anything still on the stack at the end has no greater element → −1.
Time / Space	O(n) — each element is pushed and popped once. O(n) stack + map.

SECTION 03

Recursion

A function that solves a problem by calling itself on a smaller piece of it.

What is Recursion?

Recursion is a technique where a function calls itself to solve smaller sub-problems.

- A function may call itself directly, or call another function that calls it back.
- Every recursion needs a base case — a condition that stops the calls.
- Without a base case, the recursion never ends and the call stack overflows.

SECTION 03 · RECURSION

23

Recursion Example — Factorial

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0)            // base case
        return 1;
    return n * factorial(n - 1);   // recursive step
}

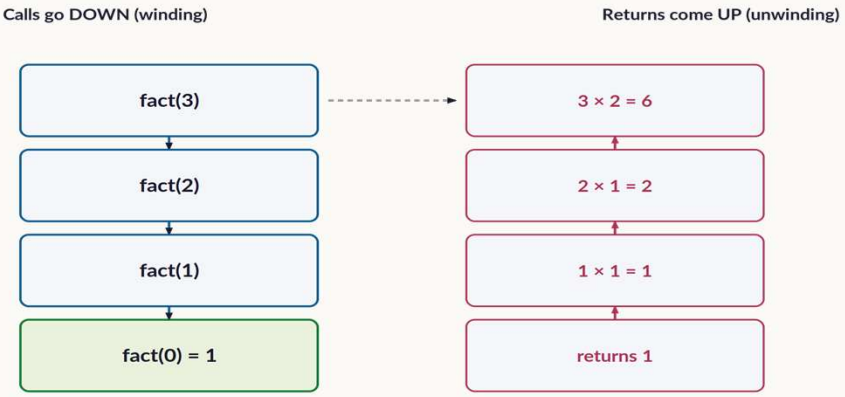
int main() {
    cout << factorial(3);  // prints 6
}
```

SECTION 03 · RECURSION

24

How Recursion Uses the Stack

Calls stack up (winding); the base case triggers returns that unwind back to the answer.



Each call waits on the one below it; the base case (fact(0)=1) stops the recursion and answers unwind back up.

SECTION 03 · RECURSION

25

Types of Recursion

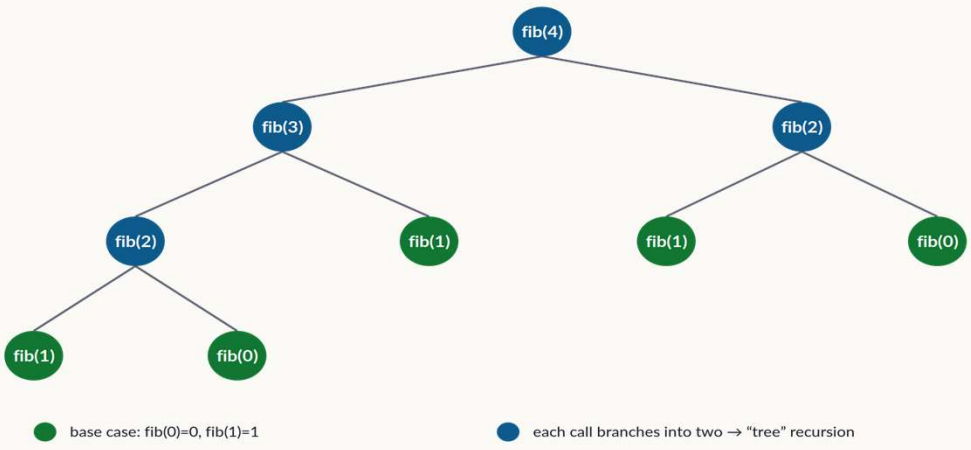
Direct	A function calls itself directly.
Indirect	Function A calls B, and B calls A back — a circular chain.
Tail	The recursive call is the last statement; nothing happens after it returns.
Head	Work happens after the recursive call returns (not tail).
Tree	A function calls itself more than once per call (branching).
Nested	A recursive call is passed as an argument to another recursive call.

SECTION 03 · RECURSION

26

Example — Fibonacci (Tree Recursion)

Each fib call branches into two — a clear picture of tree recursion.



fib(4) recomputes fib(2) twice and fib(1) three times — simple, but exponential without memoization.

SECTION 03 · RECURSION

Fibonacci — Code & Cost

Steps

1. Base case: fib(0)=0, fib(1)=1
2. Else return fib(n−1) + fib(n−2)
3. Two calls per level → tree shape
4. Recomputes sub-problems → exponential
5. Memoization caches results → O(n)

```
// pseudocode
int fib(int n) {
    if (n < 2)              // base case
        return n;
    return fib(n-1) + fib(n-2);
}
// fib(4) = fib(3) + fib(2)
// simple but O(2^n) without
// memoization (caching results)
```

SECTION 04

Backtracking

Try a choice; if it dead-ends, undo it and try another.

SECTION 04 · BACKTRACKING

30

What is Backtracking?

Build a solution step by step; if a step leads to a dead end, undo it and try a different one.

- A recursive problem-solving technique that explores options incrementally.
- When an option can't lead to a valid solution, it “backtracks” to the last choice.
- Used for mazes, puzzles (Sudoku, N-Queens), and generating combinations.

SECTION 04 · BACKTRACKING

31

General Outline

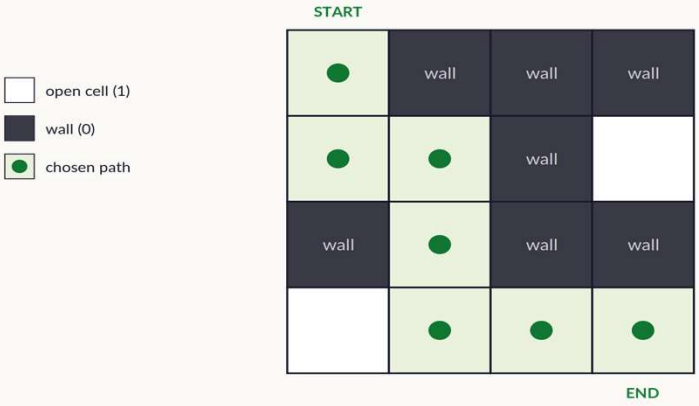
- Choose an initial (partial) solution.
- Explore all possible extensions of the current solution.
- If an extension reaches a full valid solution → return it.
- If an extension leads to a dead end → undo it (backtrack) and try the next option.

SECTION 04 · BACKTRACKING

32

Example — Rat in a Maze

The rat tries a direction; on a dead end it backtracks and tries another, until it reaches the exit.



Try a direction; if it dead-ends, backtrack and try another. The rat explores until it reaches the exit.

SECTION 04 · BACKTRACKING

33

Rat in a Maze — Approach

Steps

1. If current cell is the destination → success
2. If cell is out of bounds or a wall → return false
3. Mark the cell as part of the path
4. Try moving (down, right, up, left) recursively
5. If none work → unmark (backtrack), return false

```
// pseudocode
bool solve(maze, r, c):
    if (r,c) == dest: return true
    if not isValid(maze, r, c):
        return false
    path[r][c] = 1            // choose
    for each (dr,dc) in moves:
        if solve(maze, r+dr, c+dc):
            return true
    path[r][c] = 0            // un-choose
    return false              // backtrack
```

SECTION 05

Practice It Yourself

Stack problems are pattern-recognition — build the pattern by hand first.

PRACTICE · STACKS

P1

Problems to Try

Trace each one on paper with a real stack drawn beside it before you write code.

LeetCode 20	**Valid Parentheses**	Easy	Check if brackets are balanced using a stack.
LeetCode 155	**Min Stack**	Easy	Design a stack that returns its minimum in O(1).
LeetCode 150	**Evaluate RPN**	Medium	Evaluate a postfix (reverse-Polish) expression.
LeetCode 682	**Baseball Game**	Easy	Process operations on a stack of scores.
LeetCode 739	**Daily Temperatures**	Medium	Classic monotonic-stack warm-up.

PRACTICE · WORKED EXAMPLE

P2

LeetCode 20 — The Approach

Problem (in short): given a string of brackets ()[]{} , decide if every bracket is correctly opened and closed.

- Scan the string left to right, keeping a stack of “expected” closers.
- On an opening bracket → push its matching closing bracket.
- On a closing bracket → it must equal what we pop; if not, it's invalid.
- At the end, a valid string leaves the stack empty.

YOUR TURN

On paper, trace "([ ] { })" and then "( ] ". Watch the stack grow and shrink — where does the second one fail?

PRACTICE · WORKED EXAMPLE

P3

LeetCode 20 — The Code

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(') st.push(')');
        else if (c == '[') st.push(']');
        else if (c == '{') st.push('}');
        else {                          // closing bracket
            if (st.empty() || st.top() != c)
                return false;
            st.pop();
        }
    }
    return st.empty();                  // all matched?
}
```

PRACTICE · WORKED EXAMPLE

P4

Why It Works & How Fast

The idea	Pushing the expected closer means a match is just an equality check when we pop.
Catches errors	A wrong closer, or a closer with an empty stack, fails immediately.
Ends empty	Left-over openers mean unmatched brackets → not valid.
Time / Space	O(n) time (one pass), O(n) space (worst case all openers).

PRACTICE · RESOURCES

P5

Where to Get Help

NeetCode	Valid Parentheses, stack pattern. Search: "NeetCode Valid Parentheses"
Abdul Bari	Stacks & recursion fundamentals. Search: "Abdul Bari Stack"
William Fiset	Stack & recursion playlists. Search: "William Fiset Stack data structure"

YOUR TURN

Course rule: try each problem for 20 minutes and trace the stack on paper before watching anything — and never paste it into an AI.

LECTURE 4 — RECAP

What we covered

- Stack = LIFO: push, pop, peek at the top
- Implemented with arrays or linked lists
- Postfix evaluation with a single stack pass
- Recursion: base case + recursive step
- Types: direct/indirect, tail/head, tree/nested
- Backtracking: choose, explore, undo (rat in a maze)

Next: Queues